

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Debugging & Optimization Task

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 2 – Debugging & Optimisation Task
Weightage:	35% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	PRENA M	Reg. No.:	192524056
Section / Batch:	A	Date:	22/07/2026

Code of Conduct

I PRENA M [192524056]

(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: PRENA M

Instructions

- This test contains 20 Debugging & Optimisation Task questions. Total: 100 Marks (20*5).
- When a student answer using Scenario-Based, in evaluation the answer should be with following five requirements: Identify the problem type and constraints, Select appropriate data structure, Design the Solution, Optimization and efficiency, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Search Data Structure, Stack	Binary Search, Stack Operations	1 - 5	25
Linked List Data Structure	List Operations	6 - 10	25
Queue Data Structures	Queue Operations	11 - 15	25
Queue Data Structures, Stack Notations	Queue Operations, Expression Evaluation	16 - 20	25
GRAND TOTAL:		100 Marks	

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Debugging Accuracy	20	All errors corrected; code runs flawlessly	Most errors corrected; minor issues remain	Partial correction; code runs with issues	Code fails or major errors persist
Error Identification	30	Accurately identifies all logical, syntax, and edge-case errors	Identifies most major errors	Identifies some errors but misses key issues	Struggles to identify errors
Code Quality & Readability	20	Clean, modular, well documented, follows best practices	Mostly clean and readable	Some structure but inconsistent	Poorly written, hard to understand
Explanation & Justification	20	Clear, logical, and well justified reasoning with complexity analysis	Good explanation with some justification	Limited explanation; lacks depth	No or incorrect explanation
Testing & Validation	10	Comprehensive test cases including edge cases	Adequate test cases	Minimal testing	No testing provided
Faculty In-charge		Course Coordinator		Program Director	
Signature: _____		Signature: _____		Signature: _____	
Date: _____		Date: _____		Date: _____	

SET – 1

Your Fixed Code

```
1 #include <stdio.h>
2
3 int binarySearch(int arr[], int n, int key)
4 {
5     int low = 0, high = n - 1;
6
7     while (low <= high)
8     {
9         int mid = (low + high) / 2;
10
11         if (arr[mid] == key)
12             return mid;
13
14         else if (arr[mid] < key)
15             low = mid + 1;
16
17         else
18             high = mid - 1;
19     }
20
21     return -1;
22 }
23
```

Test Input

```
10 20 30 40 50
30
```

Output

```
Element found at position 3
```

⌚ Pending manual review

Your Fixed Code

```
1 #include <stdio.h>
2
3 int stack[5];
4 int top = -1;
5
6 int pop()
7 {
8     if (top == -1)
9     {
10         printf("Underflow");
11         return -1;
12     }
13
14     return stack[top--];
15 }
16
17 int main()
18 {
19     stack[0] = 10;
20     stack[1] = 20;
21     stack[2] = 30;
22     top = 2;
```

Test Input

```
10 20 30
2
```

Output

⌚ Pending manual review

Your Fixed Code

```
1 #include <stdio.h>
2
3 int stack[5];
4 int top;
5
6 void init()
7 {
8     top = -1;
9 }
10
11 int main()
12 {
13     init();
14     printf("Stack initialized successfully.\n");
15     printf("Top = %d", top);
16
17     return 0;
18 }
```

✓ Submitted

Test Input

stdin input (optional)

Output

```
Stack initialized successfully.
Top = -1
```

⌚ Pending manual review

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <ctype.h>
4 #include <string.h>
5
6 #define MAX 100
7
8 char stack[MAX];
9 int top = -1;
10
11 // Push function
12 void push(char item) {
13     if (top >= MAX - 1) {
14         printf("Stack Overflow\n");
15         return;
16     }
17     stack[++top] = item;
18 }
19
```

Test Input

stdin input (optional)

Output

```
Infix Expression: a*(b+c)-d/e
Postfix Expression: abc+*de/-
```

⌚ Pending manual review

Your Fixed Code

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 100
5
6 char stack[MAX];
7 int top = -1;
8
9 // Push function
10 void push(char ch) {
11     if (top < MAX - 1) {
12         stack[++top] = ch;
13     }
14 }
15
16 // Pop function
17 void pop() {
18     if (top >= 0) {
19         top--;
20     }
21 }
22
```

Test Input

stdin input (optional)

Output

Balanced

⌚ Pending manual review

Q6 Linked List

5 marks

What is the issue?
temp = head
while temp.next != NULL:
print(temp.data)
temp = temp.next

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 int main()
11 {
12     struct Node *head = NULL, *temp, *newNode;
13     int n, i, value;
14 }
```

Test Input

5
10 20 30 40 50

Output

10 20 30 40 50

Q7 Stack

5 marks

Point out the error.

POP(stack):

if top == 0:

print("Underflow")

else:

return stack[top--]

✳ **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 100
4
5 int stack[MAX];
6 int top = -1;
7
8 int pop()
9 {
10     if (top == -1)
11     {
12         printf("Underflow");
13         return -1;
14     }
```

Test Input

3
10 20 30

Output

30

Q8 Linked List Data Structures

5 marks

Identify the errors in the following code

while current != NULL:

current.next = prev

prev = current

current = current.next

✳ **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 int main()
11 {
12     struct Node *head = NULL, *temp, *newNode;
13     struct Node *prev = NULL, *current, *next;
14     int n, i, value;
15 }
```

Test Input

5
10 20 30 40 50

Output

50 40 30 20 10

Q9 Linked List

5 marks

What is the error in the below code?

```
new.next = head
head = new
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 int main()
11 {
12     struct Node *head = NULL;
13     struct Node *newNode, *temp;
14     int n, i, value;
15
16     scanf("%d", &n);
17
18     for (i = 0; i < n; i++)
```

Test Input

```
3
10 20 30
```

Output

```
30 20 10
```

Q10 Queue

5 marks

What is missing in the below pseudocode?

```
DEQUEUE(Q):
if front == -1:
    print("Underflow")
else:
    return Q[front++]
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 100
4
5 int Q[MAX];
6 int front = -1, rear = -1;
7
8 void enqueue(int value)
9 {
10     if (rear == MAX - 1)
11         printf("Overflow\n");
12     else
13     {
14         if (front == -1)
15             front = 0;
```

Test Input

```
3
10 20 30
```

Output

```
10
```

SET-2

Your Fixed Code

```
1 #include <stdio.h>
2
3 int stack[100], top = -1;
4
5 void push(int x) {
6     stack[++top] = x;
7 }
8
9 int pop() {
10     return stack[top--];
11 }
12
13 int main() {
14     char expression[100];
15     int i, a, b, result;
16
17     scanf("%99s", expression);
18
19     for (i = 0; expression[i] != '\0'; i++) {
20         if (expression[i] >= '0' && expression[i] <= '9') {
21             push(expression[i] - '0');
22         }
23         else {
```

Test Input

23^54*+9-

Output

11

Q2 Circular Queue

5 marks

Identify the bug in this Circular Queue dequeue logic.

```
int dequeue() {
    if (front == -1) return -1;
    int data = queue[front];
    front = (front + 1) % MAX;
    return data;
}
```

🔍 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int queue[MAX];
6 int front = -1, rear = -1;
7
8 int dequeue() {
9     if (front == -1)
10         return -1;
11
12     int data = queue[front];
13 }
```

Test Input

10

Output

10

Q3 Binary Search

5 marks

Identify the bug in this Recursive Binary Search.

```
int binarySearch(int arr[], int l, int r, int x) {
    int mid = l + (r - l) / 2;
    if (arr[mid] == x) return mid;
    if (arr[mid] > x) return binarySearch(arr, l, mid, x);
    return binarySearch(arr, mid, r, x);
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 int binarySearch(int arr[], int l, int r, int x) {
4     if (l > r)
5         return -1;
6
7     int mid = l + (r - l) / 2;
8
9     if (arr[mid] == x)
10        return mid;
11
12    if (arr[mid] > x)
13        return binarySearch(arr, l, mid - 1, x);
```

Test Input

2

Output

2

Q4 Queue

5 marks

Find the error in this Deque insertFront operation.

```
void insertFront(int data) {
    if (front == 0) front = MAX - 1;
    else front = front - 1;
    deque[front] = data;
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int deque[MAX];
6 int front = -1, rear = -1;
7
8 void insertFront(int data) {
9     if (front == -1) {
10        front = rear = 0;
11    } else if (front == 0) {
12        front = MAX - 1;
13    } else {
14        front = front - 1;
15    }
```

Test Input

10

Output

10

Q5 Queue

5 marks

Identify the error in the Queue enqueue logic.

```
void enqueue(int data) {
    if (rear == MAX) return;
    queue[rear++] = data;
}
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int queue[MAX];
6 int rear = -1;
7
8 void enqueue(int data) {
9     if (rear == MAX - 1)
10         return;
11
12     queue[++rear] = data;
13 }
14
15 int main() {
16     enqueue(10);
```

Test Input

```
10
20
30
```

Output

```
10 20 30
```

Q6 Doubly Linked List

5 marks

Identify the error in this Doubly Linked List node deletion.

```
void deleteNode(Node* del) {
    if (del->prev != NULL)
        del->prev->next = del->next;
    if (del->next != NULL)
        del->next->prev = del->prev;
}
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *prev;
7     struct Node *next;
8 };
9
10 void deleteNode(struct Node **head, struct Node *del) {
11     if (*head == NULL || del == NULL)
12         return;
13 }
```

Test Input

```
10
20
30
```

Output

```
10 30
```

Q7 Stack

5 marks

Identify the issue in the below pseudocode.

POP():

if top == 0 → Underflow

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX];
6 int top = -1;
7
8 int pop() {
9     if (top == -1) {
10         printf("Underflow");
11         return -1;
12     }
13
14     return stack[top--];
15 }
16
17 int main() {
18     stack[++top] = 10;
```

Test Input

10 20

Output

20

Q8 Stack

5 marks

Point out the error.

if top == MAX:

Overflow

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX];
6 int top = -1;
7
8 void push(int data) {
9     if (top == MAX - 1) {
10         printf("Overflow");
11         return;
12     }
13
14     stack[++top] = data;
15 }
16
17 int main() {
18     push(10);
```

Test Input

10 20

Output

20

Q9 Circular Queue

5 marks

Identify the issue in the following snippet for Circular Queue in Traffic Signal.

```
rear = (rear + 1) % MAX;
queue[rear] = vehicle;
if(rear == front){
    printf("Queue Full");
}
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int queue[MAX];
6 int front = -1, rear = -1;
7
8 void enqueue(int vehicle) {
9     if ((rear + 1) % MAX == front) {
10         printf("Queue Full");
11         return;
12     }
13
14     if (front == -1) {
```

Test Input

stdin input (optional)

Output

101 102 103

Q10 Stack

5 marks

Point out the error?

```
PEEK(stack):
return stack[top+1]
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX];
6 int top = -1;
7
8 int peek() {
9     if (top == -1) {
10         printf("Underflow");
11         return -1;
12     }
13
14     return stack[top];
15 }
16
17 int main() {
18     stack[++top] = 10;
```

Test Input

stdin input (optional)

Output

20